Problema 1 (4.5pt)

Consideremos un sistema no lineal de la forma $\mathbf{f}(\mathbf{x}) = \mathbf{0}$. Se desea implementar una modificación del método de Newton–Raphson que consiste en calcular la matriz jacobiana $\mathbb{J} := \partial\mathbf{f}/\partial\mathbf{x}$ únicamente cada K iteraciones, siendo K un dato, y mantener fijo su valor en las iteraciones intermedias. Es decir, las iteraciones $k = 0, K, 2K, \ldots$ son de la forma habitual

$$\begin{aligned} \mathbb{F}^k &:= \mathbb{J}(\mathbf{x}^k), \\ \mathbf{p}^k &:= -\left(\mathbb{F}^k\right)^{-1} \mathbf{f}^k, \\ \mathbf{x}^{k+1} &:= \mathbf{x}^k + \mathbf{p}^k, \end{aligned}$$

mientras que el resto de iteraciones reutilizan la matriz jacobiana calculada en la iteración anterior,

$$\begin{aligned} \mathbb{F}^k &:= \mathbb{F}^{k-1}, \\ \mathbf{p}^k &:= -\left(\mathbb{F}^k\right)^{-1} \mathbf{f}^k, \\ \mathbf{x}^{k+1} &:= \mathbf{x}^k + \mathbf{p}^k. \end{aligned}$$

Cuestión 1 (1pt): *Indicar las ventajas e inconvenientes que presenta el método anterior.*

Solución: Con el método modificado, evitamos el cálculo de la matriz jacobiana en cada iteración, que suele ser muy costoso. Asimismo, durante K iteraciones, la matriz del sistema que determina $\mathbf{p}^k$ siempre es la misma, por lo que puede reutilizarse su factorización, que también suele ser muy costosa ($\mathcal{O}\left(N^3\right)$ si la matriz es llena), para reducir aún más el coste computacional. (El hecho de que se puede reutilizar la factorización es clave y es el aspecto a señalar más importante de este problema.) Por tanto, el coste por iteración será mucho menor. Como inconveniente, al no utilizar la matriz jacobiana exacta, la convergencia es más lenta (en particular, es lineal en lugar de cuadrática), por lo que se necesitarán más iteraciones.

No es cierto que el método "pierda precisión", ni que dé unos errores mayores que el método de Newton–Raphson original. La precisión es la misma (la tolerancia establecida), la diferencia es que necesita un número mayor de iteraciones.

Tampoco es cierto que ocupe menos ni más memoria. El método de Newton–Raphson no almacena en memoria *todas* las matrices jacobianas, lo cual sería absolutamente inviable; en su lugar, almacena la de la iteración actual, que es la única que necesita.

En lo que sigue, se supone que $\mathbb{J}$ es una matriz llena (es decir, no dispersa) y no simétrica.

Cuestión 2 (3pt): *Implementar una función*

```
[x_k, nIters, flag] = NewtonModif(fun, x0, Tol, MaxIter, K),
```

donde los argumentos de entrada y de salida, a excepción de K, son los mismos que los de la función `NewtonRaphson1`*, que implemente el método descrito arriba de forma eficiente.*

Solución: Para decidir si se actualiza la jacobiana y su factorización, se puede utilizar un contador, o bien hallar el resto de la división entera entre el número de iteraciones y el valor de K. Además, puesto que la matriz jacobiana se supone llena y no simétrica, su factorización más adecuada es la LU con pivotaje parcial. Véase la función `NewtonModif1`.

Cuestión 3 (0.5pt): *Implementar una función* `Polin_NewtonModif(N,K)` *que, recibidos los valores de N y K, resuelva el sistema polinomial (sección 6.3 de los apuntes) utilizando el método implementado en* `NewtonModif`*. Fijando $N = 4000$, completar la tabla 1 con el número de iteraciones y los tiempos empleados según los distintos valores de K. Justificar los resultados.*

Solución: Véase la función `Polin_NewtonModif1(N,K)`. La tabla 1 muestra los resultados para un procesador i7-8750H @2.20GHz[1]. Como puede verse, conforme K es mayor, el número de iteraciones crece, como cabe esperar de acuerdo a lo comentado anteriormente. Sin embargo, al evaluar menos veces la matriz jacobiana, y especialmente al reutilizar su factorización, cada iteración resulta mucho menos costosa, de tal forma que el tiempo de cálculo total disminuye.

Tabla 1: Número de iteraciones y tiempos de cálculo según el valor de K.

K	Número de iteraciones	Tiempo de resolución del sistema [s]
1	5	2.87
2	6	1.94
5	11	1.65
10	15	1.21

[1]Estos datos son de carácter informativo; no era necesario dar información sobre el procesador en el examen.

Problema 2 (5.5pt)

Una pared de espesor $L = 0.1\,m$ separa un líquido a temperatura $T_0 = 400\,K$ de un ambiente a temperatura $T_\infty = 298\,K$. La pared posee un coeficiente de conductividad térmica dependiente de la temperatura $k(T) = k_0(T_0/T)^2$, con $k_0 = 0.2\,W/(m \cdot K)$. El extremo de la pared en contacto con el líquido está siempre a la temperatura T_0, mientras que en el extremo en contacto con el ambiente existe un fenómeno de convección natural de coeficiente $h = 50\,W/(m^2K)$.

Se desea hallar la distribución de temperaturas T en el interior de la pared, así como el flujo q de calor por unidad de superficie transmitido al ambiente. Se supone que la temperatura depende únicamente de una coordenada longitudinal x que recorre el interior de la pared. Para ello, se definen N nodos equiespaciados $x_1, \ldots, x_N$ a lo largo de dicha coordenada, de tal forma que $x_1 = 0$ y $x_N = L$ coinciden con los extremos de la pared en contacto con el líquido y el ambiente, véase la figura 1. Asimismo, se denota por $T_1, \ldots, T_N$ a las temperaturas en dichos nodos, es decir, $T_i = T(x_i)$. En ese caso, es posible demostrar (bajo ciertas aproximaciones que son más precisas cuanto mayor es N) que se satisface el sistema no lineal de ecuaciones

$$f_1(\mathbf{u}) := T_1 - T_0 \qquad\qquad = 0, \tag{1}$$

$$f_i(\mathbf{u}) := k_i(\mathbf{u})\frac{T_i - T_{i-1}}{\Delta x} + q = 0, \quad i = 2, \ldots, N, \tag{2}$$

$$f_{N+1}(\mathbf{u}) := -h\,(T_N - T_\infty) + q \;= 0, \tag{3}$$

donde

$$k_i(\mathbf{u}) := k\left(\frac{T_{i-1} + T_i}{2}\right) = k_0\left(\frac{2T_0}{T_{i-1} + T_i}\right)^2,$$

$\Delta x := L/(N-1)$ y $\mathbf{u} = [T_1, \ldots, T_N, q]^{\mathrm{T}}$ es un vector con las incógnitas a hallar.

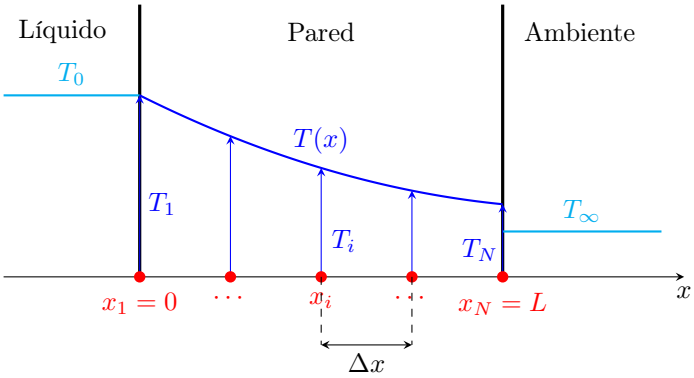

Figura 1: Esquema de la pared y de la distribución de temperatura a lo largo de la misma.

Para facilitar los cálculos, en lo que sigue se despreciarán las derivadas de $k_i(\mathbf{u})$ respecto de los elementos en $\mathbf{u}$. Es decir, a la hora de calcular derivadas, se puede tratar $k_i(\mathbf{u})$ como si fuera una constante.

Cuestión 4 (0.5pt): *Hallar los elementos no nulos de la matriz jacobiana* $\mathbb{J}_{ij}(\mathbf{u}) := \partial f_i(\mathbf{u})/\partial u_j$, *donde, como es habitual,* u_j *denota el* j*-ésimo elemento de* $\mathbf{u}$. *Utilícese una notación que deje claro qué posición ocupan dichos elementos en la matriz* $\mathbb{J}$.

Solución:

$$J_{11} = 1,$$

$$J_{i,i-1} = -\frac{k_i(\mathbf{u})}{\Delta x}, \qquad J_{i,i} = \frac{k_i(\mathbf{u})}{\Delta x}, \qquad J_{i,N+1} = 1, \qquad i = 2, \ldots, N,$$

$$J_{N+1,N} = -h, \qquad J_{N+1,N+1} = 1.$$

Cuestión 5 (2.5pt): *Implementar una función* `[f,J] = ParedResiduo(u, CalcJ)` *que, recibido el vector* $\mathbf{u}$ *y una variable booleana* `CalcJ`*, devuelva el residuo* $\mathbf{f}(\mathbf{u}) = [f_1(\mathbf{u}), \ldots, f_{N+1}(\mathbf{u})]^{\mathrm{T}}$*. Además, si* `CalcJ` *tiene valor* `true`*, la variable* `J` *debe ser la matriz jacobiana* $\mathbb{J}(\mathbf{u})$*, en caso contrario, simplemente* `J=NaN`*.*

Solución: Es fundamental observar que la matriz jacobiana es dispersa. Se considera error grave construirla como matriz llena. Incluso aunque luego se construya la matriz con la orden `sparse`, tampoco se puede utilizar en ningún momento la orden `zeros(N+1,N+1)`, puesto que ésta genera una matriz llena de dimensiones $(N+1)\times(N+1)$, que es precisamente lo que queremos evitar. Véanse las funciones `ParedResiduo1` y `ParedResiduo2`.

En lugar del sistema original (1)-(3), se considera el sistema precondicionado

$$\mathbf{g}(\mathbf{u}) := \frac{1}{2}\mathbb{L}^{-1}\mathbf{f}(\mathbf{u}) + \frac{1}{2}\mathbb{U}^{-1}\mathbf{f}(\mathbf{u}) = \mathbf{0}, \tag{4}$$

donde $\mathbb{L}$ y $\mathbb{U}$ son respectivamente las partes triangular inferior y superior de la matriz $\mathbb{J}(\mathbf{u}^0)$, mientras que $\mathbf{u}^0$ es la aproximación inicial, que se especificará más adelante. Se supone que $\mathbb{L}^{-1} + \mathbb{U}^{-1}$ es una matriz no singular.

En lo que sigue, puede resultar útil la función `linspace(a, b, N)`, que genera un vector fila con N elementos equiespaciados entre a y b. Asimismo, las partes triangular inferior y superior de una matriz $\mathbb{A}$ pueden extraerse respectivamente con los comandos `tril(A)` y `triu(A)`.

Cuestión 6 (1.25pt): *Implementar una función* `Problema2(N)` *que, recibido el valor de* N*, realice las siguientes tareas:*

- *Resolver el sistema* (4) *mediante el método de Anderson. Tómese como aproximación inicial* $T^0(x_i)$ *para la temperatura una distribución lineal con* x_i *tal que* $T^0(x_1) = T_0$ *y* $T^0(x_N) = T_\infty$*, y una aproximación inicial* $q^0 = 0$ *para el flujo de calor. Asimismo, tómese una tolerancia de* 10^{-6}*, un máximo de 100 iteraciones y 10 condiciones secantes.*
- *Mostrar por pantalla el flujo de calor* q*, así como la temperatura en el extremo en contacto con el ambiente,* $T(x_N)$*.*
- *Representar la distribución de temperaturas* $T(x_i)$ *frente a la coordenada* x_i *con una línea azul continua y con puntos (opción* `'.-b'` *del comando* `plot`*).*

Solución: Véase la función `Problema2_2026PEP1.`

Cuestión 7 (1.25pt): *Justificar la elección del residuo precondicionado* (4)*.*

Solución: Para utilizar el método de Anderson, el residuo precondicionado debe satisfacer tres condiciones:

(i) Resolver $\mathbf{f}(\mathbf{u}) = \mathbf{0}$ equivale a resolver $\mathbf{g}(\mathbf{u}) = \mathbf{0}$. En efecto, a partir de (4), se tiene

$$\mathbf{g}(\mathbf{u}) = \frac{1}{2}\left(\mathbb{L}^{-1} + \mathbb{U}^{-1}\right)\mathbf{f}(\mathbf{u}).$$

Es inmediato ver que, si $\mathbf{f}(\mathbf{u}) = \mathbf{0}$, entonces $\mathbf{g}(\mathbf{u}) = \mathbf{0}$. De la misma forma, puesto que de acuerdo al enunciado $\mathbb{L}^{-1} + \mathbb{U}^{-1}$ es no singular, $\mathbf{g}(\mathbf{u}) = \mathbf{0}$ implica $\mathbf{f}(\mathbf{u}) = \mathbf{0}$.

(ii) Evaluar $\mathbf{g}(\mathbf{u})$ no es mucho más costoso que evaluar $\mathbf{f}(\mathbf{u})$. Notemos que las matrices $\mathbb{L}$ y $\mathbb{U}$ son fijas y triangulares (y, además, son muy fáciles de calcular una vez se tiene $\mathbb{J}(\mathbf{u}^0)$). Por tanto, una vez conocido $\mathbf{f}(\mathbf{u})$, hallar $\mathbf{g}(\mathbf{u})$ sólo requiere resolver dos sistemas triangulares, lo cual es una operación relativamente poco costosa.

(iii) La matriz $\partial \mathbf{g}/\partial \mathbf{u} \approx \mathbb{I}$. En efecto,

$$\frac{\partial \mathbf{g}}{\partial \mathbf{u}}(\mathbf{u}) = \frac{1}{2}\mathbb{L}^{-1}\mathbb{J}(\mathbf{u}) + \frac{1}{2}\mathbb{U}^{-1}\mathbb{J}(\mathbf{u}).$$

Puesto que $\mathbb{L}$ y $\mathbb{U}$ son respectivamente las partes triangular superior y triangular inferior de $\mathbb{J}(\mathbf{u}^0)$, podemos suponer

$$\mathbb{L} \approx \mathbb{J}(\mathbf{u}^0), \quad \mathbb{U} \approx \mathbb{J}(\mathbf{u}^0),$$

de tal modo que

$$\frac{\partial \mathbf{g}}{\partial \mathbf{u}}(\mathbf{u}) \approx \frac{1}{2}\mathbb{J}^{-1}(\mathbf{u}^0)\mathbb{J}(\mathbf{u}) + \frac{1}{2}\mathbb{J}^{-1}(\mathbf{u}^0)\mathbb{J}(\mathbf{u}).$$

Además, si la condición inicial $\mathbf{u}^0$ está suficientemente cerca de $\mathbf{u}$, entonces

$$\mathbb{J}(\mathbf{u}) \approx \mathbb{J}(\mathbf{u}^0),$$

de tal modo que

$$\frac{\partial \mathbf{g}}{\partial \mathbf{u}}(\mathbf{u}) \approx \frac{1}{2}\mathbb{I} + \frac{1}{2}\mathbb{I} = \mathbb{I}.$$

Por supuesto, conviene recordar que todas aproximaciones se interpretan en un sentido muy heurístico, y que serán más o menos apropiadas según el problema en cuestión.